

PYTHON 2026

Apostila completa — aula por aula

Do zero ao desenvolvimento de aplicações com Python, VS Code, terminal integrado, .venv, Git e Antigravity

COMO ESTA APOSTILA FOI CONSTRUÍDA

A apostila mantém a sequência central do material-base: conceitos iniciais, primeiro programa, comentários, variáveis, leitura/saída de dados, operadores, strings, condicionais, repetição, listas, tuplas, conjuntos, dicionários, classes, módulos, banco de dados, CRUD e WebServices. O ambiente de desenvolvimento é atualizado de Replit/PyCharm para Python + VS Code + terminal integrado + .venv, e o conteúdo de IA/Antigravity é acrescentado como camada contemporânea.

Público-alvo

Alunos iniciantes de ADS, Ciência da Computação e áreas correlatas. A linguagem é apresentada de forma progressiva, com foco em prática e construção de um projeto integrador.

Princípio didático

Conceito → exemplo → execução → modificação → desafio → reflexão com IA. Cada aula termina com uma entrega pequena que prepara a próxima.

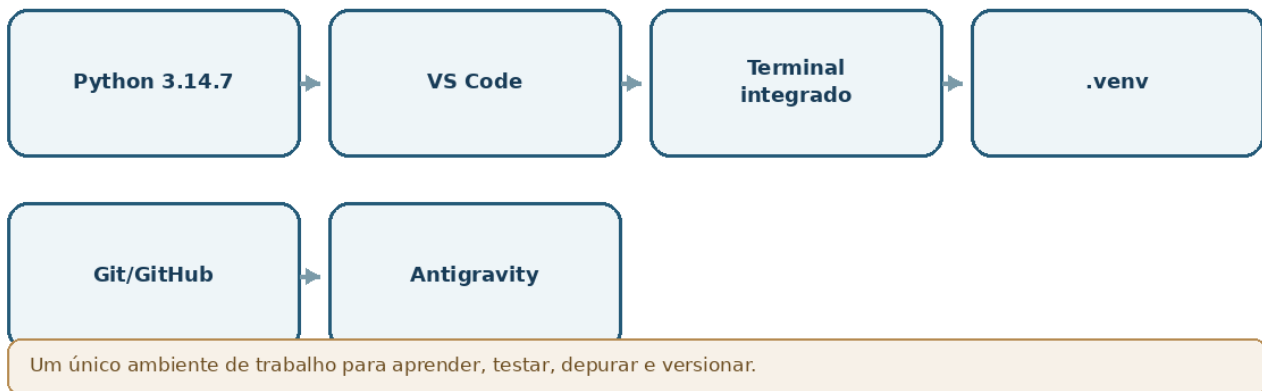
1. Visão geral da disciplina

A sequência abaixo transforma o material-base em 21 aulas. Os temas antigos não são descartados; eles são reorganizados e atualizados para um fluxo de desenvolvimento local e profissional.

Aula	Tema	Produto da aula
1	Conhecendo Python e preparando o ambiente	Ambiente funcionando
2	Primeiro programa, comentários e variáveis	Perfil do aluno
3	Entrada e saída de dados	Programa de entrada/saída
4	Operadores e expressões	Conversor numérico
5	Strings: trabalhando com textos	Texto normalizado
6	Booleanos, comparação e lógica	Regras booleanas
7	Condicional: if, elif e else	Sistema de decisão
8	Repetição com for e range()	Tabuada/iteração
9	while, break e continue	Menu com while
10	Listas: armazenando coleções	Cadastro em lista
11	Tuplas, conjuntos e operações de conjuntos	Operações com conjuntos
12	Dicionários e dados estruturados	Dados estruturados
13	Funções: reutilização e organização	Funções do cadastro
14	Módulos, imports, arquivos e exceções	Projeto modular
15	Programação Orientada a Objetos: classes e objetos	Classe Aluno
16	POO: organização, composição e boas práticas	Modelo de domínio
17	Ambiente virtual, pip, dependências e projeto profissional	Projeto com .venv
18	Git, GitHub, debugging e testes	Projeto testado/versionado
19	Banco de dados com SQLite e CRUD	CRUD SQLite
20	JSON, APIs, HTTP e WebServices	Desenho de API
21	Antigravity, projeto final e apresentação	Projeto final

2. Ambiente padrão

Ambiente da disciplina



Durante a apostila, quando aparecer “terminal”, a referência padrão será o **terminal integrado do VS Code**. Ele pode estar usando PowerShell no Windows, mas continua sendo o terminal dentro do VS Code.

ATUALIZAÇÃO EM RELAÇÃO AO MATERIAL-BASE

O material original usava inicialmente Replit e depois PyCharm. Aqui o ambiente é atualizado para VS Code + Python + extensão Python + terminal integrado + .venv. A documentação atual do VS Code trata esses componentes como peças distintas que trabalham juntas.

Comandos básicos

```
python --version
python -m pip --version
python -c "print('Olá, Python!')"
```

Ambiente virtual

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

O Python 3.14.7 documenta venv como mecanismo para criar ambientes virtuais isolados; a convenção .venv é comum em projetos. O ambiente virtual não deve ser versionado no Git.

AULA 01 — Conhecendo Python e preparando o ambiente

Objetivos de aprendizagem

- Reconhecer Python como linguagem de programação.
- Diferenciar linguagem, interpretador, editor e IDE/ambiente de desenvolvimento.
- Instalar Python, VS Code e extensões essenciais.
- Executar o primeiro comando no terminal integrado.

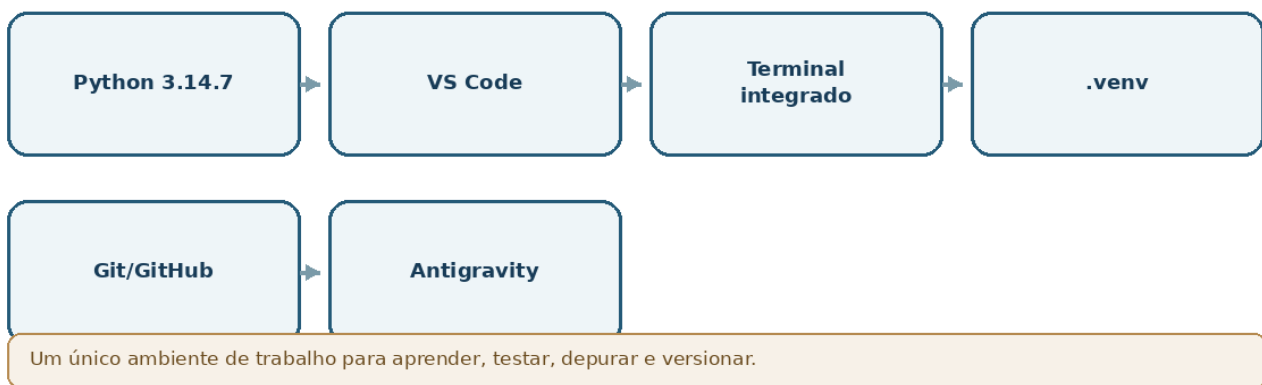
Conceitos-chave

Python e interpretador • VS Code • Terminal integrado • Extensão Python • Pylance • Python Debugger • Ruff

Explicação

O material-base começa diferenciando Python de IDE e apresenta primeiro um ambiente online e depois PyCharm. Nesta versão, o fluxo é atualizado para VS Code. O VS Code é o editor; a extensão Python acrescenta suporte de desenvolvimento; o interpretador Python executa o código.

Ambiente da disciplina



Exemplo guiado

```
python --version
python -m pip --version
python -c "print('Olá, Python!')"
```

Atividade prática

1. Verifique a versão do Python.
2. Abra o VS Code e crie uma pasta chamada aula-python.
3. Abra Terminal → Novo Terminal.
4. Execute o comando de versão.
5. Crie main.py e execute o primeiro programa.

DESAFIO

Crie um programa que imprima seu nome, curso, período e uma frase sobre o que você espera aprender em Python.

Uso pedagógico da IA

Peça ao Antigravity: “Explique para um iniciante a diferença entre Python, interpretador, VS Code e extensão Python. Não gere código.”

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 02 — Primeiro programa, comentários e variáveis

Objetivos de aprendizagem

- Usar print().
- Criar comentários.
- Declarar variáveis por atribuição.
- Identificar tipos básicos.
- Entender que nomes de variáveis representam valores usados pelo programa.

Conceitos-chave

print() • comentários # • atribuição = • int • float • str • bool • type()

Explicação

O material-base apresenta print(), comentários e variáveis como uma “caixinha” de memória. A ideia permanece útil: uma variável é um nome associado a um valor. Em Python, não é necessário declarar previamente o tipo da variável.

Exemplo guiado

```
# Meu primeiro programa
nome = "Ana"
idade = 20
altura = 1.70
estudante = True

print(nome)
print(idade)
print(type(nome))
print(type(idade))
```

Atividade prática

1. Crie quatro variáveis com dados pessoais fictícios.
2. Use type() para descobrir os tipos.
3. Altere um valor e observe o resultado.
4. Crie comentários que expliquem o código.

DESAFIO

Crie um perfil de aluno com nome, matrícula, curso, idade e média. Exiba os dados de forma organizada.

Uso pedagógico da IA

Peça uma explicação sobre a diferença entre variável e valor, sem pedir código.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 03 — Entrada e saída de dados

Objetivos de aprendizagem

- Usar `input()`.
- Converter strings para números.
- Combinar entrada e saída.
- Evitar o erro de tentar operar diretamente texto e número.

Conceitos-chave

`input()` • `int()` • `float()` • `str()` • `print()` • f-string

Explicação

O material-base apresenta leitura de dados e conversão de strings para números. Um ponto essencial para o aluno é lembrar que `input()` devolve texto. Se o programa precisa calcular, é necessário converter.

Exemplo guiado

```
nome = input("Digite seu nome: ")
idade = int(input("Digite sua idade: "))
altura = float(input("Digite sua altura: "))

print(f"Nome: {nome}")
print(f"Idade: {idade}")
print(f"Altura: {altura:.2f}")
```

Atividade prática

1. Peça dois números e calcule a soma.
2. Peça largura e altura e calcule a área.
3. Peça preço e quantidade e calcule o total.
4. Formate um valor monetário com duas casas decimais.

DESAFIO

Faça um programa que calcule o IMC a partir de peso e altura. O foco da aula é entrada, conversão, cálculo e saída.

Uso pedagógico da IA

Peça ao Antigravity para explicar por que `input()` retorna `str` e mostrar um exemplo mínimo.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 04 — Operadores e expressões

Objetivos de aprendizagem

- Usar operadores aritméticos.
- Distinguir / de // e %.
- Usar potência **.
- Construir expressões com variáveis.

Conceitos-chave

+ • - • * • / • // • % • ** • precedência

Explicação

O material-base lista soma, subtração, multiplicação, divisão, divisão inteira, módulo e potência. A aula amplia o entendimento para expressões compostas e precedência.

Exemplo guiado

```
a = 22
b = 8

print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(2 ** 3)

resultado = (a + b) * 2
```

Atividade prática

1. Calcule o preço final com desconto.
2. Calcule quantas notas de 50 cabem em um valor.
3. Use % para descobrir se um número é par.
4. Use // e % para converter minutos em horas e minutos.

DESAFIO

Faça um programa que receba uma quantidade de segundos e converta para horas, minutos e segundos.

Uso pedagógico da IA

Peça exemplos que demonstrem a diferença entre /, // e %.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 05 — Strings: trabalhando com textos

Objetivos de aprendizagem

- Criar e concatenar strings.
- Usar índices.
- Usar slicing.
- Aplicar métodos comuns de texto.
- Formatar strings com f-strings.

Conceitos-chave

str • concatenação • + • índices • slicing • lower() • upper() • strip() • replace() • split()

Explicação

O material-base trata strings junto aos primeiros fundamentos e mostra concatenação. Aqui a aula é ampliada para operações que o aluno encontrará em aplicações reais.

Exemplo guiado

```
nome = "Ana Paula"
print(nome.upper())
print(nome.lower())
print(nome[0])
print(nome[:3])
print(nome.replace("Paula", "Silva"))

curso = "ADS"
print(f"{nome} está no curso de {curso}.")
```

Atividade prática

1. Conte caracteres de um texto.
2. Converta uma frase para maiúsculas.
3. Substitua uma palavra.
4. Separe nome e sobrenome.
5. Verifique se uma palavra aparece em uma frase.

DESAFIO

Crie um programa que receba um nome completo e produza uma versão normalizada: sem espaços extras e com iniciais em maiúsculas.

Uso pedagógico da IA

Peça ao Antigravity para explicar slicing usando três exemplos diferentes.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 06 — Booleanos, comparação e lógica

Objetivos de aprendizagem

- Produzir valores True/False.
- Usar operadores de comparação.
- Combinar condições com and, or e not.
- Entender expressões booleanas.

Conceitos-chave

bool • == • != • > • < • >= • <= • and • or • not

Explicação

O material-base apresenta os operadores de comparação e negação. Nesta aula, o foco é construir a ponte entre comparação e tomada de decisão.

Exemplo guiado

```
idade = 20
possui_documento = True

maior = idade >= 18
print(maior)

pode_entrar = idade >= 18 and possui_documento
print(pode_entrar)
```

Atividade prática

1. Teste condições com números.
2. Crie uma condição de aprovação.
3. Combine idade e documento.
4. Use not para inverter uma condição.

DESAFIO

Crie uma regra de acesso a um laboratório: idade mínima de 18 anos e matrícula ativa.

Uso pedagógico da IA

Peça uma tabela-verdade simples para and, or e not e depois explique com exemplos em Python.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 07 — Condicionais: if, elif e else

Objetivos de aprendizagem

- Criar decisões simples.
- Criar decisões múltiplas.
- Compreender indentação.
- Evitar condições desnecessariamente aninhadas.

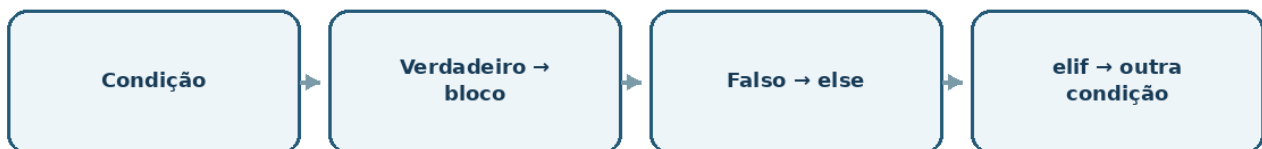
Conceitos-chave

if • elif • else • indentação • blocos

Explicação

O material-base enfatiza que os dois-pontos e o recuo definem o bloco associado ao if. Também alerta que muitos ifs aninhados podem reduzir a legibilidade. Mantemos esse cuidado.

Estrutura de decisão



A indentação determina o bloco de código.

Exemplo guiado

```
nota = float(input("Nota: "))

if nota >= 7:
    situacao = "Aprovado"
elif nota >= 5:
    situacao = "Recuperação"
else:
    situacao = "Reprovado"

print(situacao)
```

Atividade prática

1. Classifique uma idade.
2. Classifique uma nota.
3. Identifique maior entre dois números.
4. Crie um menu com opções.

DESAFIO

Faça um sistema de classificação de atendimento com prioridade normal, preferencial e urgente.

Uso pedagógico da IA

Envie um código com um erro de indentação ao Antigravity e peça somente uma explicação do erro antes da correção.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 08 — Repetição com for e range()

Objetivos de aprendizagem

- Compreender iteração.
- Usar range().
- Percorrer listas e strings.
- Evitar índices quando não são necessários.

Conceitos-chave

for • range() • iteráveis • índice • len()

Explicação

O material-base explica range(n), range(a,n) e range(a,n,s), além de mostrar for percorrendo uma lista diretamente e com índices. A recomendação didática é começar pela forma direta.

Repetição



Use for quando a iteração é natural; while quando a repetição depende de uma condição.

Exemplo guiado

```
for i in range(1, 6):  
    print(i)  
  
frutas = ["maçã", "uva", "banana"]  
for fruta in frutas:  
    print(fruta)
```

Atividade prática

1. Mostre números de 1 a 10.
2. Mostre apenas pares.
3. Faça uma tabuada.
4. Percorra uma lista de nomes.
5. Conte letras de uma palavra.

DESAFIO

Crie uma tabuada completa de 1 a 10 usando dois loops.

Uso pedagógico da IA

Peça ao Antigravity para comparar for item in lista com for i in range(len(lista)).

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 09 — while, break e continue

Objetivos de aprendizagem

- Usar while.
- Controlar encerramento.
- Usar break.
- Usar continue.
- Evitar loops infinitos.

Conceitos-chave

while • condição de parada • break • continue

Explicação

While é útil quando a quantidade de repetições não é conhecida antecipadamente. O aluno deve aprender a identificar a variável ou evento que fará a condição deixar de ser verdadeira.

Repetição



Use for quando a iteração é natural; while quando a repetição depende de uma condição.

Exemplo guiado

```
senha = ""

while senha != "python123":
    senha = input("Senha: ")

print("Acesso liberado")
```

Atividade prática

1. Crie um contador.
2. Peça números até o usuário digitar 0.
3. Use break para encerrar um menu.
4. Use continue para ignorar números negativos.

DESAFIO

Crie um menu textual que permaneça ativo até o usuário escolher “Sair”.

Uso pedagógico da IA

Peça uma análise de um while infinito e solicite que a IA identifique a causa sem alterar o código.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 10 — Listas: armazenando coleções

Objetivos de aprendizagem

- Criar listas.
- Acessar índices.
- Alterar elementos.
- Adicionar e remover itens.
- Usar slicing e len().

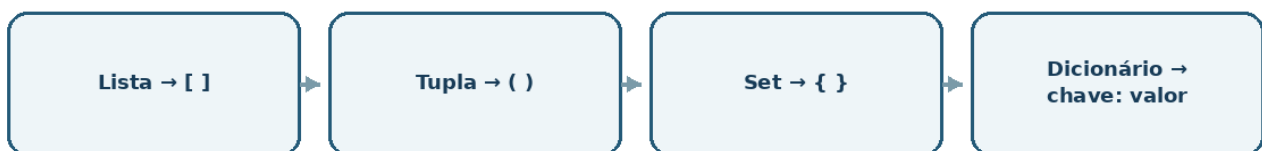
Conceitos-chave

list • índice • mutabilidade • append() • insert() • remove() • pop() • len() • slicing

Explicação

O material-base destaca que listas são mutáveis, ordenadas, permitem duplicatas e podem conter diferentes tipos. Também explica índices começando em zero e métodos de inserção.

Estruturas de dados



Escolha a estrutura conforme a forma como os dados precisam ser organizados.

Exemplo guiado

```
alunos = ["Ana", "João", "Maria"]

print(alunos[0])
alunos.append("Carlos")
alunos.insert(1, "Beatriz")
alunos.remove("João")

print(alunos)
print(len(alunos))
```

Atividade prática

1. Monte uma lista de compras.
2. Adicione três itens.
3. Remova um item.
4. Ordene a lista.
5. Percorra a lista com for.

DESAFIO

Crie um cadastro simples de alunos usando uma lista e um menu de inclusão, listagem e remoção.

Uso pedagógico da IA

Peça ao Antigravity para explicar mutabilidade de listas com um exemplo curto.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 11 — Tuplas, conjuntos e operações de conjuntos

Objetivos de aprendizagem

- Criar tuplas.
- Compreender imutabilidade.
- Criar sets.
- Eliminar duplicatas.
- Usar união, interseção e diferença.

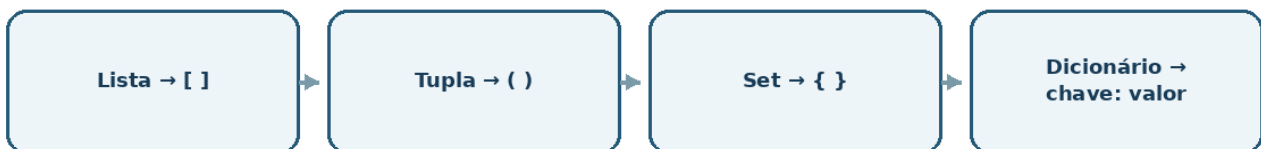
Conceitos-chave

tuple • set • imutabilidade • union • intersection • difference • remove • discard • pop

Explicação

O material-base apresenta tuplas como imutáveis e conjuntos com operações de união, interseção e diferença. A aula preserva esses conceitos e acrescenta exemplos aplicados.

Estruturas de dados



Escolha a estrutura conforme a forma como os dados precisam ser organizados.

Exemplo guiado

```
coordenada = (10, 20)
print(coordenada[0])

A = {"Python", "JavaScript", "Java"}
B = {"Python", "Java", "C#"}

print(A | B) # união
print(A & B) # interseção
print(A - B) # diferença
```

Atividade prática

1. Crie uma tupla com dados fixos.
2. Crie dois conjuntos de alunos de disciplinas diferentes.
3. Descubra alunos comuns.
4. Descubra alunos exclusivos de uma turma.

DESAFIO

Modele a participação de alunos em dois eventos e descubra quem participou dos dois, de apenas um e de nenhum.

Uso pedagógico da IA

Peça uma explicação comparando list, tuple e set em um cenário real.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 12 — Dicionários e dados estruturados

Objetivos de aprendizagem

- Criar dicionários.
- Acessar chaves.
- Atualizar valores.
- Percorrer chaves e valores.
- Criar listas de dicionários.

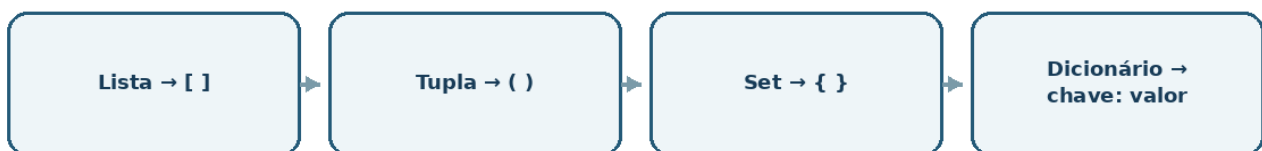
Conceitos-chave

dict • chave • valor • keys() • values() • items() • get()

Explicação

Dicionários são fundamentais para representar registros estruturados e fazem uma ponte natural com JSON e APIs. A aula prepara o aluno para armazenar entidades como aluno, produto ou usuário.

Estruturas de dados



Escolha a estrutura conforme a forma como os dados precisam ser organizados.

Exemplo guiado

```
aluno = {
    "nome": "Ana",
    "matricula": 123,
    "notas": [8, 9, 10]
}

print(aluno["nome"])
aluno["curso"] = "ADS"

for chave, valor in aluno.items():
    print(chave, valor)
```

Atividade prática

1. Crie um dicionário de produto.
2. Atualize o preço.
3. Adicione estoque.
4. Liste todas as chaves.
5. Crie uma lista de cinco alunos.

DESAFIO

Crie uma lista de dicionários representando alunos e calcule a média de cada um.

Uso pedagógico da IA

Peça ao Antigravity para transformar uma lista de dicionários em uma tabela conceitual, sem gerar código adicional.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 13 — Funções: reutilização e organização

Objetivos de aprendizagem

- Criar funções.
- Usar parâmetros.
- Retornar valores.
- Usar valores padrão.
- Aplicar type hints e docstrings.

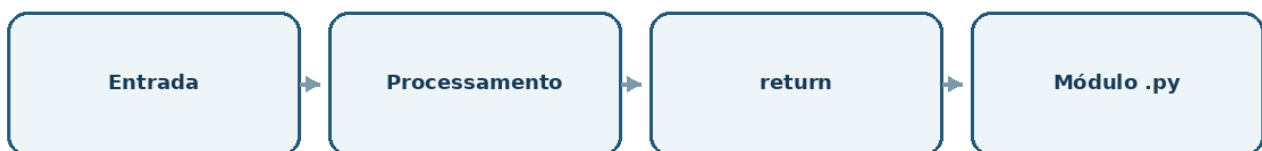
Conceitos-chave

def • parâmetros • argumentos • return • escopo • type hints • docstrings

Explicação

A partir desta aula o aluno deixa de colocar toda a lógica em um único arquivo. Funções separam responsabilidades e facilitam testes e manutenção.

Funções e módulos



Separar responsabilidades torna o programa mais legível e reutilizável.

Exemplo guiado

```
def calcular_media(notas: list[float]) -> float:
    """Calcula e retorna a média das notas."""
    return sum(notas) / len(notas)

media = calcular_media([8, 9, 10])
print(media)
```

Atividade prática

1. Crie função para somar.
2. Crie função para verificar par/ímpar.
3. Crie função para calcular desconto.
4. Crie função para validar idade.

DESAFIO

Transforme o cadastro de alunos das aulas anteriores em funções separadas: cadastrar, listar, buscar e remover.

Uso pedagógico da IA

Peça para a IA explicar por que uma função que retorna valor costuma ser mais reutilizável do que uma função que apenas imprime.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 14 — Módulos, imports, arquivos e exceções

Objetivos de aprendizagem

- Separar código em módulos .py.
- Usar import e from ... import.
- Ler e escrever arquivos.
- Tratar exceções previsíveis.

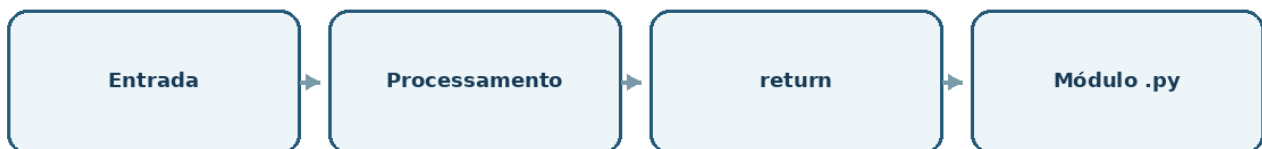
Conceitos-chave

módulos • import • from/import • open() • with • try • except • ValueError • FileNotFoundError

Explicação

O material-base apresenta módulos como arquivos .py que armazenam métodos, classes e variáveis. Também mostra a necessidade de modularizar à medida que o sistema cresce. Aqui acrescentamos arquivos e tratamento de exceções.

Funções e módulos



Separar responsabilidades torna o programa mais legível e reutilizável.

Exemplo guiado

```
# calculadora.py
def somar(a, b):
    return a + b

# main.py
from calculadora import somar

print(somar(10, 20))

try:
    idade = int(input("Idade: "))
except ValueError:
    print("Digite um número inteiro.")
```

Atividade prática

1. Separe funções em módulos.
2. Crie arquivo dados.txt.
3. Grave uma lista de nomes.
4. Leia o arquivo.
5. Trate arquivo inexistente.

DESAFIO

Refatore o projeto de alunos para uma pasta com módulos de menu, regras e dados.

Uso pedagógico da IA

Peça uma revisão de arquitetura do projeto e solicite que a IA indique responsabilidades de cada módulo, sem escrever código.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 15 — Programação Orientada a Objetos: classes e objetos

Objetivos de aprendizagem

- Criar classes.
- Criar objetos.
- Usar atributos e métodos.
- Compreender self.
- Usar `__init__`.

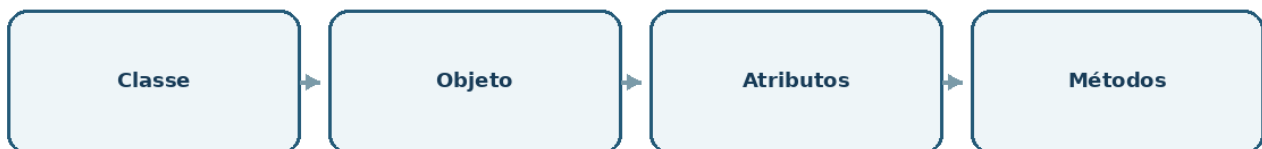
Conceitos-chave

class • objeto • instância • atributo • método • self • `__init__`

Explicação

O material-base usa a analogia da forma de bolo para explicar classe e objeto. Mantemos a analogia, mas acrescentamos um exemplo de domínio de software.

Programação Orientada a Objetos



Classe é o modelo; objeto é uma instância desse modelo.

Exemplo guiado

```
class Aluno:
    def __init__(self, nome, matricula):
        self.nome = nome
        self.matricula = matricula
        self.notas = []

    def adicionar_nota(self, nota):
        self.notas.append(nota)

    def media(self):
        return sum(self.notas) / len(self.notas)

aluno = Aluno("Ana", 1001)
aluno.adicionar_nota(8)
aluno.adicionar_nota(9)
print(aluno.media())
```

Atividade prática

1. Crie classe Produto.
2. Crie classe Conta.
3. Crie classe Livro.
4. Adicione métodos de negócio.

DESAFIO

Converta o cadastro de alunos para POO.

Uso pedagógico da IA

Peça ao Antigravity para explicar linha por linha uma classe simples, sem modificar o código.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 16 — POO: organização, composição e boas práticas

Objetivos de aprendizagem

- Diferenciar atributos de métodos.
- Usar composição.
- Evitar classes gigantes.
- Entender herança em contexto.

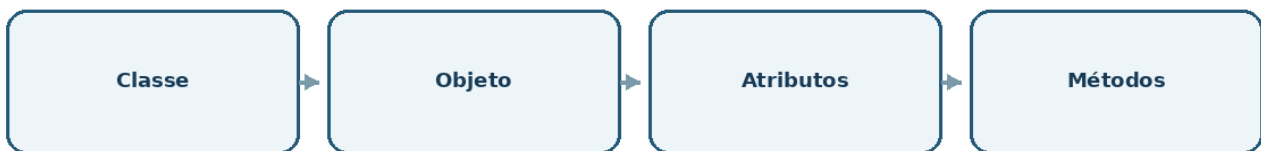
Conceitos-chave

composição • herança • polimorfismo • responsabilidade • coesão

Explicação

A orientação é não transformar POO em uma coleção de classes sem propósito. A modelagem deve representar responsabilidades do domínio. Para iniciantes, composição deve ser apresentada como alternativa natural à herança em muitos cenários.

Programação Orientada a Objetos



Classe é o modelo; objeto é uma instância desse modelo.

Exemplo guiado

```
class Endereco:
    def __init__(self, cidade):
        self.cidade = cidade

class Pessoa:
    def __init__(self, nome, endereco):
        self.nome = nome
        self.endereco = endereco

endereco = Endereco("Campina Grande")
pessoa = Pessoa("Ana", endereco)
print(pessoa.endereco.cidade)
```

Atividade prática

1. Modele Pedido e ItemPedido.
2. Modele Curso e Disciplina.
3. Identifique objetos e responsabilidades em um sistema de biblioteca.

DESAFIO

Desenhe antes de programar as classes de um sistema de biblioteca e justifique cada classe.

Uso pedagógico da IA

Peça uma crítica de modelagem: “Há classes demais? Há responsabilidades misturadas? Sugira apenas mudanças conceituais.”

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 17 — Ambiente virtual, pip, dependências e projeto profissional

Objetivos de aprendizagem

- Criar .venv.
- Ativar o ambiente.
- Instalar pacotes.
- Gerar requirements.txt.
- Compreender por que .venv não vai para o Git.

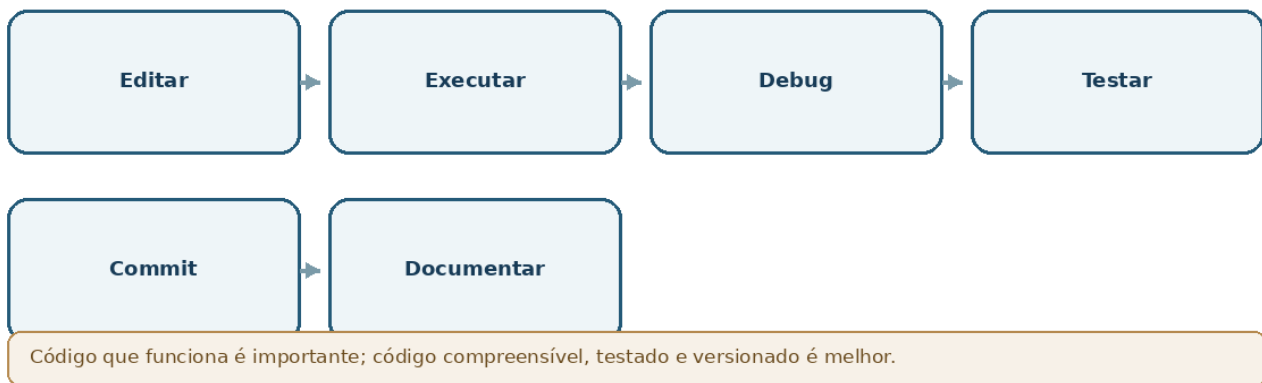
Conceitos-chave

venv • .venv • pip • requirements.txt • dependências • pyproject.toml

Explicação

A documentação atual do Python recomenda venv para ambientes virtuais. Ambientes virtuais isolam pacotes de projetos diferentes. O VS Code também consegue localizar ambientes .venv do workspace.

Fluxo profissional



Exemplo guiado

```
python -m venv .venv

# PowerShell no terminal integrado do VS Code
.venv\Scripts\Activate.ps1

python -m pip install requests
python -m pip freeze > requirements.txt
```

Atividade prática

1. Crie .venv.
2. Ative o ambiente.
3. Instale uma biblioteca pequena.
4. Gere requirements.txt.
5. Desative e reative o ambiente.

DESAFIO

Documente no README como outro aluno pode recriar o ambiente do seu projeto.

Uso pedagógico da IA

Peça uma explicação do problema que `.venv` resolve usando dois projetos com versões diferentes da mesma biblioteca.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 18 — Git, GitHub, debugging e testes

Objetivos de aprendizagem

- Criar repositório Git.
- Fazer commits significativos.
- Usar breakpoints.
- Criar testes básicos.
- Entender qualidade além de “rodar sem erro”.

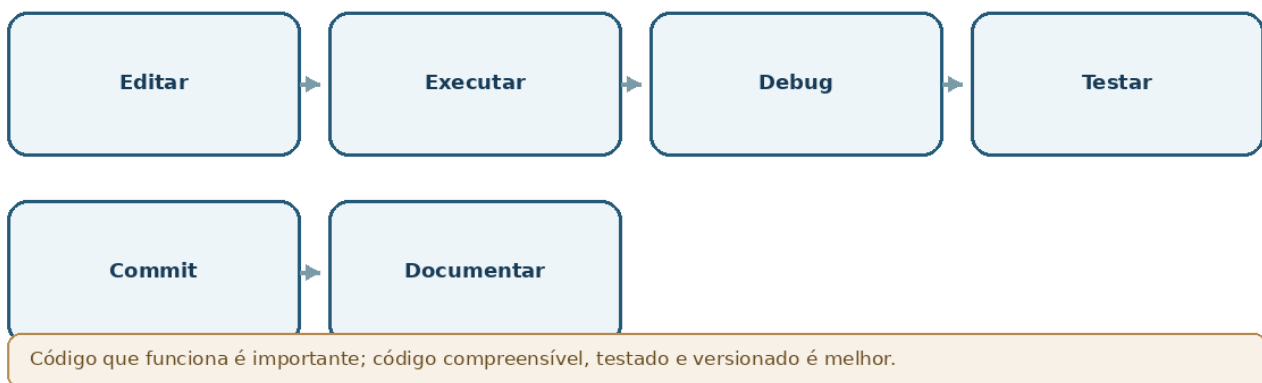
Conceitos-chave

git init • add • commit • status • README • .gitignore • debugger • pytest

Explicação

O material-base recomenda PEP 8 ao final. Nesta versão, boas práticas entram antes do projeto final. O VS Code possui suporte a execução, debugging e testes; a extensão Python oferece suporte a unittest e pytest.

Fluxo profissional



Exemplo guiado

```
git init
git add .
git commit -m "estrutura inicial"
git status
```

Atividade prática

1. Crie um repositório.
2. Faça um commit inicial.
3. Crie uma função e um teste.
4. Use breakpoint para acompanhar uma variável.
5. Documente o projeto.

DESAFIO

Pegue um programa com erro intencional, depure com breakpoint e escreva um teste que evite a regressão.

Uso pedagógico da IA

Peça ao Antigravity para sugerir casos de teste, mas escreva você mesmo pelo menos um teste.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 19 — Banco de dados com SQLite e CRUD

Objetivos de aprendizagem

- Diferenciar banco de dados e SGBD.
- Criar tabela SQLite.
- Inserir, consultar, atualizar e excluir.
- Relacionar CRUD com a aplicação Python.

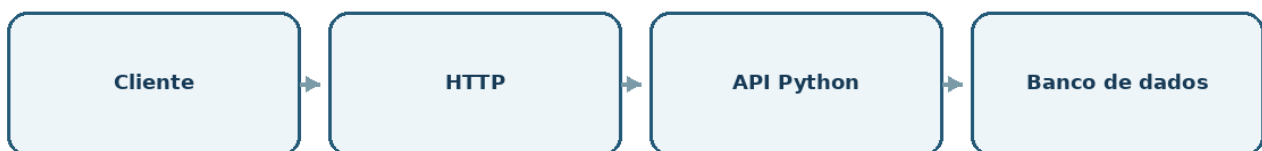
Conceitos-chave

SQLite • conexão • cursor • SQL • CREATE • INSERT • SELECT • UPDATE • DELETE • CRUD

Explicação

O material-base explica a necessidade de persistência e apresenta banco de dados como armazenamento durável. Também diferencia banco de dados de SGBD. Nesta versão, começamos com SQLite para reduzir a complexidade de instalação e conectar diretamente Python e SQL.

Arquitetura de uma API



O cliente não precisa ser Python; JSON é um formato comum de troca de dados.

Exemplo guiado

```
import sqlite3

con = sqlite3.connect("escola.db")
cur = con.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS alunos (
    id INTEGER PRIMARY KEY,
    nome TEXT NOT NULL
)
""")

cur.execute("INSERT INTO alunos (nome) VALUES (?)", ("Ana",))
con.commit()

cur.execute("SELECT id, nome FROM alunos")
for aluno in cur.fetchall():
    print(aluno)

con.close()
```

Atividade prática

1. Crie tabela alunos.
2. Insira registros.

3. Liste registros.
4. Atualize um registro.
5. Exclua um registro.

DESAFIO

Implemente o CRUD completo de alunos usando funções ou uma classe de acesso ao banco.

Uso pedagógico da IA

Peça ao Antigravity para revisar uma consulta SQL e explicar riscos de concatenar valores diretamente na string SQL.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 20 — JSON, APIs, HTTP e WebServices

Objetivos de aprendizagem

- Entender cliente e servidor.
- Conhecer GET, POST, PUT/PATCH e DELETE.
- Representar dados em JSON.
- Criar uma API Python simples.
- Testar requisições.

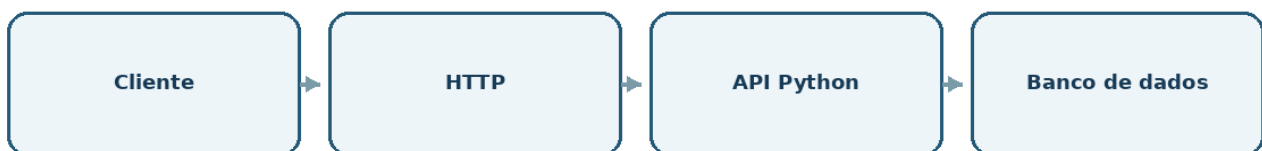
Conceitos-chave

HTTP • endpoint • GET • POST • JSON • API • cliente • servidor • status code

Explicação

O material-base introduz WebServices com Flask, verbos HTTP e JSON, destacando que clientes podem ser implementados em outras linguagens. A arquitetura é preservada, mas a aula apresenta o conceito de API de forma mais geral antes do framework.

Arquitetura de uma API



O cliente não precisa ser Python; JSON é um formato comum de troca de dados.

Exemplo guiado

```
import json

aluno = {
    "nome": "Ana",
    "curso": "ADS"
}

texto = json.dumps(aluno, ensure_ascii=False)
print(texto)
```

Atividade prática

1. Converta dicionário para JSON.
2. Leia JSON.
3. Identifique endpoints em uma API.
4. Use um cliente HTTP para testar GET e POST.
5. Observe códigos 200, 201, 400 e 404.

DESAFIO

Desenhe a API do cadastro de alunos: endpoints, métodos, entradas, respostas e códigos de status.

Uso pedagógico da IA

Peça ao Antigravity para revisar o desenho da API sem escrever a implementação.

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

AULA 21 — Antigravity, projeto final e apresentação

Objetivos de aprendizagem

- Usar IA de maneira responsável.
- Planejar uma tarefa para agente.
- Revisar diffs.
- Testar código gerado ou alterado.
- Apresentar decisões técnicas.

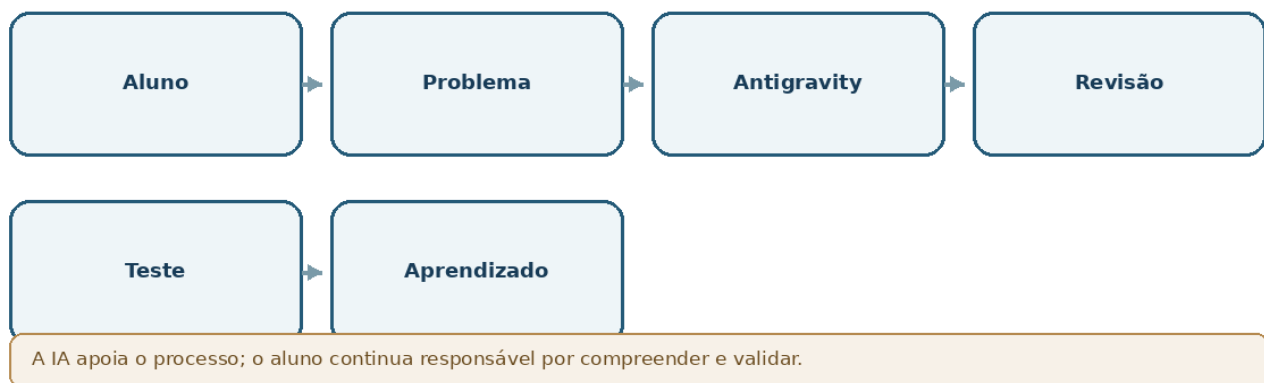
Conceitos-chave

agente • prompt • plano • diff • validação • testes • documentação • ética no uso de IA

Explicação

O Antigravity para VS Code oferece capacidades agentic dentro do editor, incluindo conversa com agentes, revisão de diffs, planos interativos e execução de tarefas de engenharia. Para a disciplina, ele será usado como ferramenta de apoio, nunca como substituto da compreensão do aluno.

IA como ferramenta de aprendizagem



Exemplo guiado

```
# Exemplo de prompt de trabalho
"Analise o projeto de cadastro de alunos.
Não altere arquivos ainda.
1. Identifique a arquitetura atual.
2. Liste problemas de organização.
3. Proponha um plano em etapas.
4. Aguarde minha aprovação antes de editar."
```

Atividade prática

1. Peça análise do projeto.
2. Solicite plano antes da implementação.
3. Revise o diff.
4. Execute testes.
5. Documente o que foi aceito e rejeitado.

DESAFIO

Entregue o projeto final com README, GitHub, testes, banco de dados ou API e registro das contribuições da IA.

Uso pedagógico da IA

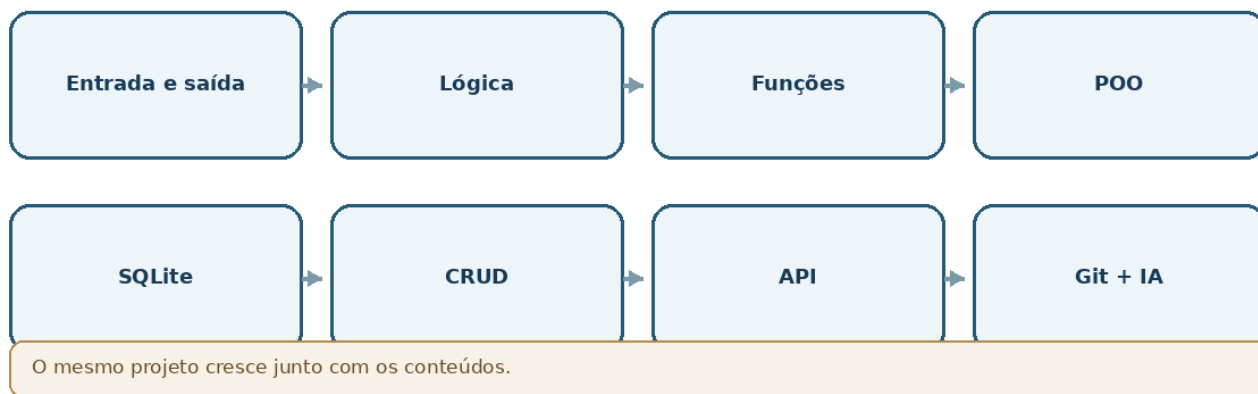
Prompt final: “Revise meu projeto como professor. Não reescreva. Liste os 5 problemas mais importantes, explique o impacto de cada um e sugira como eu mesmo posso corrigi-los.”

Checklist de saída

- Entendi o conceito
- Executei o exemplo
- Modifiquei o código
- Resolvi o desafio
- Consigo explicar o que fiz

3. Projeto Integrador — Sistema de Cadastro de Alunos

Evolução do projeto integrador



O projeto acompanha as 21 aulas e evolui incrementalmente. A cada etapa, o aluno reaproveita o que já construiu.

Etapa	Conteúdo	Entrega
1	Aulas 1–3	Programa inicial e entrada de dados
2	Aulas 4–7	Regras e classificação
3	Aulas 8–12	Coleções e cadastro
4	Aulas 13–16	Funções, módulos e POO
5	Aulas 17–18	.env, Git, testes e documentação
6	Aula 19	Persistência e CRUD SQLite
7	Aula 20	Desenho/implementação da API
8	Aula 21	IA, revisão, testes e apresentação

Requisitos mínimos de entrega

- Código organizado em módulos.
- Ambiente .env documentado.
- README com instalação e execução.
- .gitignore.
- Histórico de commits.
- Persistência em SQLite ou outra solução definida na disciplina.

- Pelo menos testes automatizados básicos.
- API ou camada de serviço documentada.
- Registro de uso da IA e validação das alterações.

CRITÉRIO CENTRAL

O aluno deve ser capaz de explicar o projeto sem depender do agente de IA. Código gerado por IA sem compreensão e validação não caracteriza domínio do conteúdo.

4. Avaliação formativa

Dimensão	Evidência
Compreensão	Explica os conceitos sem copiar respostas.
Prática	Executa e modifica exemplos.
Resolução	Resolve desafios sem solução pronta.
Qualidade	Usa nomes adequados, organização e tratamento de erros.
Testes	Demonstra comportamento esperado e casos de erro.
Versionamento	Registra evolução com commits significativos.
IA responsável	Declara uso, revisa, testa e justifica alterações.

Banco de exercícios extras

1. Calculadora de notas • 2. Conversor de unidades • 3. Caixa eletrônico • 4. Sistema de votação • 5. Agenda de contatos • 6. Cadastro de produtos • 7. Sistema de biblioteca • 8. Controle de estoque • 9. Quiz de programação • 10. API de alunos.

5. Guia rápido do professor

Para cada aula, a recomendação é trabalhar em quatro momentos: **10–15 min de retomada**, **20–30 min de explicação e demonstração**, **40–60 min de prática** e **10–15 min de socialização/reflexão**. O tempo pode ser ajustado à carga horária real da disciplina.

Perguntas de fechamento

- O que o programa faz?
- Qual dado entra?
- Qual dado sai?
- Onde está a regra principal?
- O que aconteceria se a entrada fosse inválida?
- Como você testou?
- O que você conseguiria explicar sem IA?

Checklist de preparação do laboratório

- Python instalado
- VS Code instalado
- Extensão Python
- Pylance
- Python Debugger
- Ruff
- Git
- Antigravity, quando aplicável
- Internet para instalação/documentação
- Projeto inicial distribuído aos alunos

6. Referências e atualização tecnológica

O material conceitual foi baseado no PDF fornecido pela docente. O material-base apresenta Python, variáveis, saída/entrada, operadores, condicionais, repetição, estruturas de dados, classes, módulos, banco de dados e CRUD/WebServices. ■filecite■turn3file9■L481-L542■ ■filecite■turn3file2■L135-L181■

A documentação oficial atual do Python 3.14.7 recomenda venv para ambientes virtuais e descreve a criação com `python -m venv`; ambientes virtuais isolam dependências de diferentes projetos. ■cite■turn0search1■turn0search6■

A documentação atual do VS Code diferencia o editor, a extensão Python e o interpretador, e documenta execução, debugging e testes com unittest/pytest. ■cite■turn0search8■turn0search3■

A documentação oficial do Google Antigravity descreve a extensão para VS Code e suas capacidades agentic, incluindo interação com agentes, revisão de diffs, planos e execução de tarefas. ■cite■turn0search0■turn0search2■

Recursos oficiais

Python: <https://docs.python.org/3.14/>

VS Code Python: <https://code.visualstudio.com/docs/python/python-quick-start>

Antigravity para VS Code: <https://antigravity.google/docs/ide/extensions/vscode/>